

LAPORAN PRAKTIKUM DAN TUGAS 6

Struktur Data



RAFKI AHMAD PAGAMANDA

2311533016

Kelas D

Implementasi Double Linked List

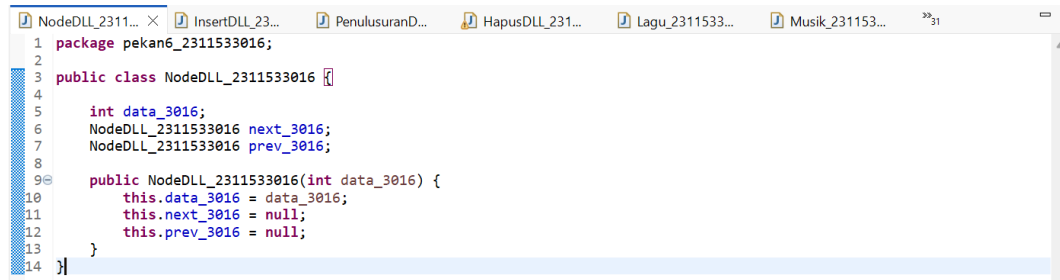
FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

Implementasi Double Linked List

1. NodeDLL_2311533016.java



```
1 package pekan6_2311533016;
2
3 public class NodeDLL_2311533016 {
4
5     int data_3016;
6     NodeDLL_2311533016 next_3016;
7     NodeDLL_2311533016 prev_3016;
8
9     public NodeDLL_2311533016(int data_3016) {
10         this.data_3016 = data_3016;
11         this.next_3016 = null;
12         this.prev_3016 = null;
13     }
14 }
```

Class NodeDLL_2311533016 merupakan class dasar yang digunakan untuk membentuk struktur Double Linked List. Pada class ini terdapat beberapa atribut penting yaitu data, pointer next, dan pointer prev. Variabel data digunakan untuk menyimpan isi node, sedangkan pointer next digunakan untuk menghubungkan node ke data berikutnya dan pointer prev digunakan untuk menghubungkan node ke data sebelumnya. Constructor pada class ini berfungsi untuk menginisialisasi data ketika objek node dibuat. Saat node baru dibuat, pointer next dan prev akan bernilai null karena node belum terhubung dengan node lain.

Tujuan dari pembuatan class ini adalah untuk menjadi pondasi utama dalam pembentukan Double Linked List. Semua proses seperti penambahan data, penghapusan data, traversal, dan pencarian data akan menggunakan node yang dibuat dari class ini. File ini tidak menghasilkan output karena hanya digunakan sebagai class pembentuk node dan tidak memiliki method main. Walaupun tidak menampilkan hasil di console, class ini sangat penting karena menjadi struktur utama pada seluruh program Double Linked List.

2. InsertDLL_2311533016.java

```

1 package pekan6_2311533016;
2
3 public class InsertDLL_2311533016 {
4
5     static NodeDLL_2311533016 insertBegin(NodeDLL_2311533016 head_3016, int data_3016) {
6
7         NodeDLL_2311533016 new_node_3016 = new NodeDLL_2311533016(data_3016);
8
9         new_node_3016.next_3016 = head_3016;
10
11         if (head_3016 != null) {
12             head_3016.prev_3016 = new_node_3016;
13         }
14
15         return new_node_3016;
16     }
17
18     public static NodeDLL_2311533016 insertEnd(NodeDLL_2311533016 head_3016, int newData_3016) {
19
20         NodeDLL_2311533016 newNode_3016 = new NodeDLL_2311533016(newData_3016);
21
22         if (head_3016 == null) {
23             head_3016 = newNode_3016;
24         } else {
25
26             NodeDLL_2311533016 curr_3016 = head_3016;
27
28             while (curr_3016.next_3016 != null) {
29                 curr_3016 = curr_3016.next_3016;
30             }
31
32             curr_3016.next_3016 = newNode_3016;
33             newNode_3016.prev_3016 = curr_3016;
34         }
35
36         return head_3016;
37     }
38
39     public static NodeDLL_2311533016 insertAtPosition(
40         NodeDLL_2311533016 head_3016,
41         int pos_3016,
42         int new_data_3016) {
43
44         NodeDLL_2311533016 new_node_3016 =
45             new NodeDLL_2311533016(new_data_3016);
46
47         if (pos_3016 == 1) {
48
49             new_node_3016.next_3016 = head_3016;
50
51             new_node_3016.next_3016 = head_3016;
52
53             if (head_3016 != null) {
54                 head_3016.prev_3016 = new_node_3016;
55             }
56
57             head_3016 = new_node_3016;
58             return head_3016;
59         }
60
61         NodeDLL_2311533016 curr_3016 = head_3016;
62
63         for (int i_3016 = 1;
64             i_3016 < pos_3016 - 1 && curr_3016 != null;
65             ++i_3016) {
66
67             curr_3016 = curr_3016.next_3016;
68         }
69
70         if (curr_3016 == null) {
71             System.out.println("Posisi tidak ada");
72             return head_3016;
73         }
74
75         new_node_3016.prev_3016 = curr_3016;
76         new_node_3016.next_3016 = curr_3016.next_3016;
77
78         curr_3016.next_3016 = new_node_3016;
79
80         if (new_node_3016.next_3016 != null) {
81             new_node_3016.next_3016.prev_3016 = new_node_3016;
82         }
83
84         return head_3016;
85     }
86
87     public static void printList(NodeDLL_2311533016 head_3016) {
88
89         NodeDLL_2311533016 curr_3016 = head_3016;
90
91         while (curr_3016 != null) {
92             System.out.print(curr_3016.data_3016 + " <-> ");
93             curr_3016 = curr_3016.next_3016;
94         }
95
96         System.out.println();
97     }
98
99     public static void main(String[] args) {

```

```

100     NodeDLL_2311533016 head_3016 =
101         new NodeDLL_2311533016(2);
102
103     head_3016.next_3016 =
104         new NodeDLL_2311533016(3);
105
106     head_3016.next_3016.prev_3016 = head_3016;
107
108     head_3016.next_3016.next_3016 =
109         new NodeDLL_2311533016(5);
110
111     head_3016.next_3016.next_3016.prev_3016 =
112         head_3016.next_3016;
113
114     System.out.print("DLL awal: ");
115     printList(head_3016);
116
117     head_3016 = insertBegin(head_3016, 1);
118
119     System.out.print("simpul 1 ditambah di awal: ");
120     printList(head_3016);
121
122     System.out.print("simpul 6 ditambah di akhir: ");
123
124     int data_3016 = 6;
125
126     head_3016 = insertEnd(head_3016, data_3016);
127
128     printList(head_3016);
129
130     System.out.print("tambah node 4 di posisi 4: ");
131
132     int data2_3016 = 4;
133     int pos_3016 = 4;
134
135     head_3016 =
136         insertAtPosition(head_3016,
137             pos_3016,
138             data2_3016);
139
140     printList(head_3016);
141 }
142 }

```

Class InsertDLL_2311533016 digunakan untuk melakukan proses penambahan node pada Double Linked List. Pada file ini terdapat beberapa method seperti method untuk menambahkan data di bagian depan linked list dan method untuk menambahkan data di bagian belakang linked list. Saat proses insert dilakukan, program akan memperbarui pointer next dan prev agar hubungan antar node tetap benar. Jika penambahan dilakukan di depan, maka node baru akan menjadi head. Sedangkan jika penambahan dilakukan di belakang, program akan menelusuri node hingga akhir lalu menghubungkan node baru di bagian terakhir.

Tujuan dari file ini adalah untuk memahami cara kerja penambahan data pada Double Linked List serta bagaimana hubungan antar node diperbarui ketika data baru dimasukkan. Selain itu, pada program juga terdapat method untuk menampilkan isi linked list sehingga pengguna dapat melihat hasil perubahan data setelah proses insert dilakukan.

Output program akan menampilkan isi linked list sebelum dan sesudah penambahan node. Misalnya data awal adalah 1 <-> 2 <-> 3, kemudian setelah ditambahkan node di depan menjadi 0 <-> 1 <-> 2 <-> 3. Setelah itu jika ditambahkan node di belakang maka hasilnya menjadi 0 <-> 1 <-> 2 <-> 3 <-> 4. Dari hasil tersebut dapat dilihat bahwa proses penambahan node berhasil dilakukan dengan baik.

Secara keseluruhan, file ini menunjukkan cara kerja proses insert pada Double Linked List baik di bagian depan maupun belakang linked list.

3. PenelusuranDLL_2311533016.java

```
1 package pekan6_2311533016;
2
3 public class PenelusuranDLL_2311533016 {
4
5     static void forwardTraversal(NodeDLL_2311533016 head_3016) {
6
7         NodeDLL_2311533016 curr_3016 = head_3016;
8
9         while (curr_3016 != null) {
10
11             System.out.print(curr_3016.data_3016 + " <-> ");
12
13             curr_3016 = curr_3016.next_3016;
14         }
15
16         System.out.println();
17     }
18
19     static void backwardTraversal(NodeDLL_2311533016 tail_3016) {
20
21         NodeDLL_2311533016 curr_3016 = tail_3016;
22
23         while (curr_3016 != null) {
24
25             System.out.print(curr_3016.data_3016 + " <-> ");
26
27             curr_3016 = curr_3016.prev_3016;
28         }
29
30         System.out.println();
31     }
32
33     public static void main(String[] args) {
34
35         NodeDLL_2311533016 head_3016 =
36             new NodeDLL_2311533016(1);
37
38         NodeDLL_2311533016 second_3016 =
39             new NodeDLL_2311533016(2);
40
41         NodeDLL_2311533016 third_3016 =
42             new NodeDLL_2311533016(3);
43
44         head_3016.next_3016 = second_3016;
45         second_3016.prev_3016 = head_3016;
46
47         second_3016.next_3016 = third_3016;
48         third_3016.prev_3016 = second_3016;
49
50         System.out.println("Penelusuran maju:");
51         forwardTraversal(head_3016);
```

Class PenelusuranDLL_2311533016 digunakan untuk melakukan traversal atau penelusuran data pada Double Linked List. Traversal dilakukan untuk menampilkan seluruh isi linked list dari awal hingga akhir maupun dari akhir hingga awal. Pada file ini terdapat method traversal maju yang menggunakan pointer next dan traversal mundur yang menggunakan pointer prev.

Tujuan dari file ini adalah untuk memahami cara kerja penelusuran data pada Double Linked List serta membuktikan bahwa struktur data ini dapat diakses dari dua arah. Hal tersebut menjadi salah satu kelebihan Double Linked List dibandingkan Single Linked List.

Ketika program dijalankan, data linked list akan ditampilkan secara berurutan dari depan ke belakang, misalnya 1 2 3 4 5. Setelah itu program juga akan menampilkan data dari belakang ke depan menjadi 5 4 3 2 1. Output tersebut menunjukkan bahwa pointer next dan prev telah terhubung dengan benar sehingga traversal dua arah dapat dilakukan tanpa error.

Secara keseluruhan, file ini menjelaskan bagaimana proses traversal pada Double Linked List dilakukan menggunakan pointer next dan prev.

4. HapusDLL_2311533016.java

```
1 package pekan6_2311533016;
2
3 public class HapusDLL_2311533016 {
4
5     public static NodeDLL_2311533016 delHead_3016(
6         NodeDLL_2311533016 head_3016) {
7
8         if (head_3016 == null) {
9             return null;
10        }
11
12        NodeDLL_2311533016 temp_3016 = head_3016;
13        head_3016 = head_3016.next_3016;
14
15        if (head_3016 != null) {
16            head_3016.prev_3016 = null;
17        }
18
19        return head_3016;
20    }
21
22    public static NodeDLL_2311533016 dellast(
23        NodeDLL_2311533016 head_3016) {
24
25        if (head_3016 == null) {
26            return null;
27        }
28
29        if (head_3016.next_3016 == null) {
30            return null;
31        }
32
33        NodeDLL_2311533016 curr_3016 = head_3016;
34
35        while (curr_3016.next_3016 != null) {
36            curr_3016 = curr_3016.next_3016;
37        }
38
39        if (curr_3016.prev_3016 != null) {
40            curr_3016.prev_3016.next_3016 = null;
41        }
42
43        return head_3016;
44    }
45
46    public static NodeDLL_2311533016 delPos_3016(
47        NodeDLL_2311533016 head_3016,
48        int pos_3016) {
49
50        if (head_3016 == null) {
51            return head_3016;
52        }
53
54        NodeDLL_2311533016 curr_3016 = head_3016;
55
56        for (int i_3016 = 1;
57            curr_3016 != null && i_3016 < pos_3016;
58            ++i_3016) {
59            curr_3016 = curr_3016.next_3016;
60        }
61
62        if (curr_3016 == null) {
63            return head_3016;
64        }
65
66        if (curr_3016.prev_3016 != null) {
67            curr_3016.prev_3016.next_3016 =
68                curr_3016.next_3016;
69        }
70
71        if (curr_3016.next_3016 != null) {
72            curr_3016.next_3016.prev_3016 =
73                curr_3016.prev_3016;
74        }
75
76        if (head_3016 == curr_3016) {
77            head_3016 = curr_3016.next_3016;
78        }
79
80        return head_3016;
81    }
82
83    public static void printList(NodeDLL_2311533016 head_3016) {
84
85        NodeDLL_2311533016 curr_3016 = head_3016;
86
87        while (curr_3016 != null) {
88            System.out.print(curr_3016.data_3016 + " ");
89            curr_3016 = curr_3016.next_3016;
90        }
91
92        System.out.println();
93    }
94
95    public static void main(String[] args) {
96
97    }
98
99 }
```

```

100
101     NodeDLL_2311533016 head_3016 =
102         new NodeDLL_2311533016(1);
103
104     head_3016.next_3016 =
105         new NodeDLL_2311533016(2);
106
107     head_3016.next_3016.prev_3016 =
108         head_3016;
109
110     head_3016.next_3016.next_3016 =
111         new NodeDLL_2311533016(3);
112
113     head_3016.next_3016.next_3016.prev_3016 =
114         head_3016.next_3016;
115
116     head_3016.next_3016.next_3016.next_3016 =
117         new NodeDLL_2311533016(4);
118
119     head_3016.next_3016.next_3016.next_3016.prev_3016 =
120         head_3016.next_3016.next_3016;
121
122     head_3016.next_3016.next_3016.next_3016.next_3016 =
123         new NodeDLL_2311533016(5);
124
125     head_3016.next_3016.next_3016.next_3016.next_3016.prev_3016 =
126         head_3016.next_3016.next_3016.next_3016;
127
128     System.out.print("DLL Awal: ");
129     printList(head_3016);
130
131     System.out.print("Setelah head dihapus: ");
132     head_3016 = delHead_3016(head_3016);
133     printList(head_3016);
134
135     System.out.print("Setelah node terakhir dihapus: ");
136     head_3016 = delLast(head_3016);
137     printList(head_3016);
138
139     System.out.print("Menghapus posisi ke-2: ");
140     head_3016 = delPos_3016(head_3016, 2);
141     printList(head_3016);
142 }
143

```

Class HapusDLL_2311533016 digunakan untuk melakukan penghapusan node pada Double Linked List. Pada file ini terdapat beberapa method yaitu method untuk menghapus node pertama, method untuk menghapus node terakhir, dan method untuk menghapus node pada posisi tertentu. Saat node dihapus, pointer next dan prev akan diperbarui agar linked list tetap tersambung dengan benar.

Tujuan dari file ini adalah untuk memahami proses delete pada Double Linked List dan bagaimana cara menjaga hubungan antar node setelah penghapusan dilakukan. Program juga mengajarkan cara menangani kondisi tertentu seperti linked list kosong atau linked list hanya memiliki satu node.

Output program akan menampilkan isi linked list sebelum dan sesudah penghapusan. Contohnya data awal adalah 1 <-> 2 <-> 3 <-> 4 <-> 5. Setelah node pertama dihapus, hasil menjadi 2 <-> 3 <-> 4 <-> 5. Setelah node terakhir dihapus menjadi 2 <-> 3 <-> 4. Kemudian jika node posisi tertentu dihapus maka data akan berubah sesuai posisi yang dipilih.

Secara keseluruhan, file ini menunjukkan cara kerja penghapusan data pada Double Linked List serta pentingnya memperbarui pointer next dan prev agar struktur linked list tetap benar.

5. Lagu_2311533016.java

```
1 package pekan6_2311533016;
2
3 public class Lagu_2311533016 {
4
5     String judul_3016;
6     String penyanyi_3016;
7
8     Lagu_2311533016 next_3016;
9     Lagu_2311533016 prev_3016;
10
11 public Lagu_2311533016(String judul_3016, String penyanyi_3016) {
12
13     this.judul_3016 = judul_3016;
14     this.penyanyi_3016 = penyanyi_3016;
15
16     this.next_3016 = null;
17     this.prev_3016 = null;
18 }
19
20 public String getJudul_3016() {
21     return judul_3016;
22 }
23
24 public String getPenyanyi_3016() {
25     return penyanyi_3016;
26 }
27
28 public void setJudul_3016(String judul_3016) {
29     this.judul_3016 = judul_3016;
30 }
31
32 public void setPenyanyi_3016(String penyanyi_3016) {
33     this.penyanyi_3016 = penyanyi_3016;
34 }
35 }
```

Class Lagu_2311533016 digunakan sebagai node untuk menyimpan data lagu pada playlist musik. Pada class ini terdapat atribut judul lagu, nama penyanyi, pointer next, dan pointer prev. Constructor digunakan untuk menginisialisasi data lagu saat objek dibuat.

Tujuan dari file ini adalah untuk membentuk struktur data playlist musik menggunakan konsep Double Linked List. Setiap lagu akan disimpan dalam bentuk node dan saling terhubung menggunakan pointer next dan prev. File ini tidak menghasilkan output karena hanya berfungsi sebagai class penyimpanan data lagu. Walaupun tidak memiliki method main, class ini sangat penting karena digunakan oleh class Musik_2311533016 untuk menjalankan seluruh fitur playlist musik.

Secara keseluruhan, file ini berfungsi sebagai pondasi utama dalam pembentukan sistem playlist musik berbasis Double Linked List.

6. Musik_2311533016.java

```
1 package pekan6_2311533016;
2
3 import java.util.Scanner;
4
5 public class Musik_2311533016 {
6
7     Lagu_2311533016 head_3016;
8     Lagu_2311533016 tail_3016;
9
10    // tambah lagu di akhir
11    public void tambahLagu_3016(String judul_3016, String penyanyi_3016) {
12
13        Lagu_2311533016 laguBaru_3016 =
14            new Lagu_2311533016(judul_3016, penyanyi_3016);
15
16        if (head_3016 == null) {
17            head_3016 = laguBaru_3016;
18            tail_3016 = laguBaru_3016;
19        } else {
20            tail_3016.next_3016 = laguBaru_3016;
21            laguBaru_3016.prev_3016 = tail_3016;
22            tail_3016 = laguBaru_3016;
23        }
24        System.out.println("Lagu berhasil ditambahkan!");
25    }
26
27    // hapus lagu pertama
28    public void hapusLaguAwal_3016() {
29
30        if (head_3016 == null) {
31            System.out.println("Playlist kosong!");
32            return;
33        }
34
35        System.out.println("Lagu "
36            + head_3016.judul_3016
37            + " berhasil dihapus");
38
39        head_3016 = head_3016.next_3016;
40
41        if (head_3016 != null) {
42            head_3016.prev_3016 = null;
43        } else {
44            tail_3016 = null;
45            tail_3016 = null;
46        }
47    }
48
49    // tampil maju
50    public void tampilMaju_3016() {
51
52        if (head_3016 == null) {
53            System.out.println("Playlist kosong!");
54            return;
55        }
56
57        Lagu_2311533016 curr_3016 = head_3016;
58
59        System.out.println("\n== Playlist Maju ==");
60
61        while (curr_3016 != null) {
62            System.out.println(
63                curr_3016.judul_3016
64                + " - "
65                + curr_3016.penyanyi_3016);
66            curr_3016 = curr_3016.next_3016;
67        }
68
69    }
70
71    // tampil mundur
72    public void tampilMundur_3016() {
73
74        if (tail_3016 == null) {
75            System.out.println("Playlist kosong!");
76            return;
77        }
78
79        Lagu_2311533016 curr_3016 = tail_3016;
80
81        System.out.println("\n== Playlist Mundur ==");
82
83        while (curr_3016 != null) {
84            System.out.println(
85                curr_3016.judul_3016
86                + " - "
87                + curr_3016.penyanyi_3016);
88            curr_3016 = curr_3016.prev_3016;
89        }
90    }
91}
```

```

100
101
102
103 // cari lagu
104 public void cariLagu_3016(String judulCari_3016) {
105     Lagu_2311533016 curr_3016 = head_3016;
106
107     while (curr_3016 != null) {
108
109         if (curr_3016.judul_3016
110             .equalsIgnoreCase(judulCari_3016)) {
111
112             System.out.println("Lagu ditemukan!");
113             System.out.println(
114                 curr_3016.judul_3016
115                 + " - "
116                 + curr_3016.penyanyi_3016);
117
118             return;
119         }
120
121         curr_3016 = curr_3016.next_3016;
122     }
123
124     System.out.println("Lagu tidak ditemukan!");
125 }
126
127 public static void main(String[] args) {
128
129     Scanner input_3016 = new Scanner(System.in);
130
131     Musik_2311533016 playlist_3016 =
132         new Musik_2311533016();
133
134     int pilihan_3016;
135
136     do {
137
138         System.out.println("\n=== Playlist Musik NIM: 2311533016 ===");
139         System.out.println("1. Tambah Lagu");
140         System.out.println("2. Hapus Lagu Pertama");
141         System.out.println("3. Lihat Playlist (Maju)");
142         System.out.println("4. Lihat Playlist (Mundur)");
143         System.out.println("5. Cari Lagu");
144         System.out.println("6. Keluar");
145
146         System.out.print("Pilihan: ");
147         pilihan_3016 = input_3016.nextInt();
148         input_3016.nextLine();
149
150         switch (pilihan_3016) {
151             input_3016.nextLine();
152
153             playlist_3016.tambahLagu_3016(
154                 judul_3016,
155                 penyanyi_3016);
156
157             break;
158
159             case 2:
160
161                 playlist_3016.hapusLaguAwal_3016();
162
163                 break;
164
165             case 3:
166
167                 playlist_3016.tampilMaju_3016();
168
169                 break;
170
171             case 4:
172
173                 playlist_3016.tampilMundur_3016();
174
175                 break;
176
177             case 5:
178
179                 System.out.print("Masukkan judul lagu: ");
180
181                 String cari_3016 =
182                     input_3016.nextLine();
183
184                 playlist_3016.cariLagu_3016(cari_3016);
185
186                 break;
187
188             case 6:
189
190                 System.out.println("Program selesai");
191                 break;
192
193             default:
194
195                 System.out.println("Pilihan tidak tersedia");
196
197         }
198
199     } while (pilihan_3016 != 6);
200
201     input_3016.close();
202 }
203
204
205
206
207
208
209
210
211

```

Class Musik_2311533016 merupakan program utama yang digunakan untuk mengelola playlist musik menggunakan Double Linked List. Pada file ini terdapat berbagai method seperti method untuk menambah lagu, menghapus lagu awal, menampilkan playlist secara maju, menampilkan playlist secara mundur, dan mencari lagu berdasarkan judul.

Tujuan dari program ini adalah untuk menerapkan konsep Double Linked List ke dalam studi kasus nyata yaitu playlist musik. Dengan adanya pointer next dan prev, playlist dapat ditampilkan dari depan maupun dari belakang dengan mudah. Saat program dijalankan, pengguna akan melihat menu pilihan seperti tambah lagu, hapus lagu, tampil maju, tampil mundur, dan cari lagu. Pengguna dapat memasukkan data lagu berupa judul dan penyanyi. Setelah data dimasukkan, playlist akan tersimpan dalam linked list.

Output program akan menampilkan daftar lagu secara maju maupun mundur. Misalnya ketika pengguna menambahkan lagu “Evaluasi” dan “To The Bone”, maka program akan menampilkan playlist maju dari lagu pertama ke terakhir dan playlist mundur dari lagu terakhir ke pertama. Selain itu program juga dapat mencari lagu berdasarkan judul yang dimasukkan pengguna.

Secara keseluruhan, file ini menunjukkan penerapan konsep Double Linked List dalam program playlist musik sederhana dengan berbagai fitur dasar pengelolaan data.

Kesimpulan

Pada praktikum pekan 6 ini dipelajari konsep Double Linked List menggunakan bahasa Java. Double Linked List memiliki dua pointer yaitu next dan prev sehingga data dapat diakses dari dua arah.

Melalui praktikum ini dapat dipahami cara:

- Menambahkan node.
- Menghapus node.
- Menelusuri data maju dan mundur.
- Menghubungkan node menggunakan pointer.
- Menerapkan Double Linked List pada studi kasus playlist musik.